

Working with Eclipse

From PostgreSQL wiki

[Jump to navigation](#)[Jump to search](#)

Contents

- 1 Developing with Eclipse
- 2 Install version control software
 - 2.1 Install git
 - 2.2 Install CVS
- 3 Open terminal
- 4 Install PostgreSQL source code =
 - 4.1 Download with git
 - 4.2 Download PostgreSQL source code - CVS
 - 4.3 Copy with CVS
 - 4.4 Use CVS
- 5 Configure source code
 - 5.1 Install components with Software-Center
 - 5.2 Configure in the terminal
- 6 Download Eclipse
 - 6.1 Install JDK for Eclipse
 - 6.2 Download Eclipse
 - 6.3 Extract and start
 - 6.4 Set up Workspace
 - 6.5 The program
- 7 Import
- 8 Adding make targets
 - 8.1 Open the Make Target view
 - 8.2 Add Target
- 9 Launch initdb in the terminal
- 10 Run Configurations in Eclipse
- 11 Add include files
- 12 Debugging PostgreSQL from Eclipse
- 13 Debugging with child processes
- 14 Create and open Database
- 15 Terminate the program

Developing with Eclipse

This page explains how to get quickly started if you want to use Eclipse to develop PostgreSQL.

Used software for this manual:

OS: *Ubuntu 8.4*

Editor: Eclipse Helios

Versioning: CVS

The manual has been tested on an Ubuntu Live Version to make sure that no required Software is already installed.

!!Important: An internet connection is required

(Remark: If you haven't installed Ubuntu yet, we recommend the following Link: <http://www.ubuntu.com/desktop/get-ubuntu/download>)

Install version control software

The preferred way to get PostgreSQL source code is to use the git version control software. This tutorial will also cover using the older cvs program. You only need one program, git or cvs, not both.

Install git

Start Ubuntu Software Center:

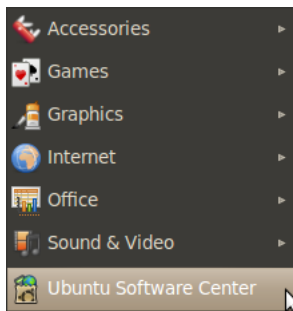
Applications » Ubuntu Software Center

To install git you should go to the magnifier on the right top side of the application, and enter "**git**". From the results choose "**fast, scalable, distributed revision control**" and click on the **Install** button (if git is not already installed).

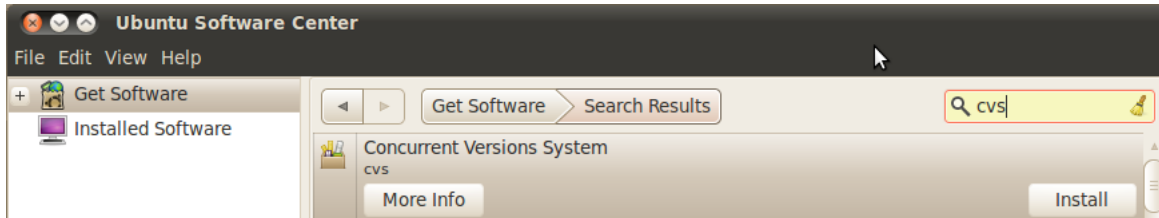
Install CVS

Start Ubuntu Software Center:

Applications » Ubuntu Software Center

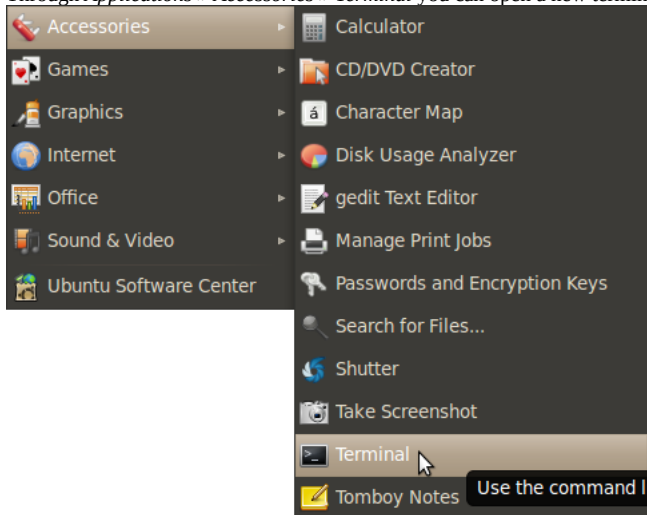


To install CVS you should go to the magnifier on the right top side of the application, and enter "**cvs**". From the results choose "**Concurrent Versions System**" and click on the **Install** button (if CVS is not already installed).



Open terminal

Through *Applications » Accessories » Terminal* you can open a new terminal:



Install PostgreSQL source code =

Now you can download the latest version of the source code, so you can edit it locally in your computer. Again you need to follow one of the cvs or git instruction sets, but not both.

Download with git

In the previous terminal, if you haven't closed it (otherwise repeat step **1. Open terminal**, and don't close it until the end of the installation), you can make a copy with the following commands:

```
mkdir project
cd project
git clone git://git.postgresql.org/git/postgresql.git
mv postgresql pgsql
```

You can find more detailed information about setting up your own repository at Working with git. The cloned directory here is renamed to "pgsql" to match the cvs examples below. You can leave it as postgresql instead, and just modify the rest of the examples below to use that name.

Download PostgreSQL source code - CVS

In our example the CVSROOT is **pgrepo** (you can choose also an other name), and is located under Home

The command `mkdir` creates a new folder (**pgrepo**) and makes a new environment variable that links to this folder:

```
mkdir pgrepo
export CVSROOT=$HOME/pgrepo
```

The next command downloads with `rsync` the source code. This can take several minutes:

```
rsync --progress -avzCH --delete anoncvs.postgresql.org::pgsql-cvs $CVSR00T
```

```

ubuntu@ubuntu: ~
File Edit View Terminal Help
17808 100% 67.15kB/s 0:00:00 (xfer#7457, to-check=10/8520)
pgsql/src/win32/
pgsql/src/win32/Attic/
pgsql/src/win32/Attic/README,v
808 100% 2.61kB/s 0:00:00 (xfer#7458, to-check=8/8520)
pgsql/src/win32/Attic/endian.h,v
1402 100% 4.52kB/s 0:00:00 (xfer#7459, to-check=7/8520)
pgsql/src/win32/Attic/ipc.patch,v
9215 100% 29.60kB/s 0:00:00 (xfer#7460, to-check=6/8520)
pgsql/src/win32/Attic/tcp.h,v
1035 100% 3.32kB/s 0:00:00 (xfer#7461, to-check=5/8520)
pgsql/src/win32/Attic/un.h,v
1653 100% 5.31kB/s 0:00:00 (xfer#7462, to-check=4/8520)
sup/
sup/pgsql/
sup/pgsql/checkouts.cvs
503032 100% 767.57kB/s 0:00:00 (xfer#7463, to-check=1/8520)
sup/repository/
sup/repository/checkouts.cvs
588002 100% 467.61kB/s 0:00:01 (xfer#7464, to-check=0/8520)

sent 146070 bytes received 123949722 bytes 154636.50 bytes/sec
total size is 569241152 speedup is 4.59
ubuntu@ubuntu:~$

```

Copy with CVS

With CVS (Concurrent Version Control) you can make a versioned copy of the folder **pgrepo**.

Use CVS

In the previous terminal, if you haven't closed it (otherwise repeat step **1. Open terminal**, and don't close it until the end of the installation), you can make a copy with the following commands:

```
mkdir project
cd project
cvs co pgsql
```

Warning: Execute these commands one layer above pgrepo, for instance in Home in our example!

```

ubuntu@ubuntu: ~/project
File Edit View Terminal Help
cvs checkout: Updating pgsql/src/tools/pgindent
U pgsql/src/tools/pgindent/README
U pgsql/src/tools/pgindent/exclude_file_patterns
U pgsql/src/tools/pgindent/indent.bsd.patch
U pgsql/src/tools/pgindent/pgcippindent
U pgsql/src/tools/pgindent/pgindent
U pgsql/src/tools/pgindent/typedefs.list
cvs checkout: Updating pgsql/src/tools/pgindent.dir
cvs checkout: Updating pgsql/src/tools/thread
cvs checkout: Updating pgsql/src/tutorial
U pgsql/src/tutorial/Makefile
U pgsql/src/tutorial/README
U pgsql/src/tutorial/advanced.source
U pgsql/src/tutorial/basics.source
U pgsql/src/tutorial/complex.c
U pgsql/src/tutorial/complex.source
U pgsql/src/tutorial/funcs.c
U pgsql/src/tutorial/funcs.source
U pgsql/src/tutorial/funcs_new.c
U pgsql/src/tutorial/syscat.source
cvs checkout: Updating pgsql/src/tutorial/C-code
cvs checkout: Updating pgsql/src/utils
cvs checkout: Updating pgsql/src/win32
ubuntu@ubuntu:~/project$

```

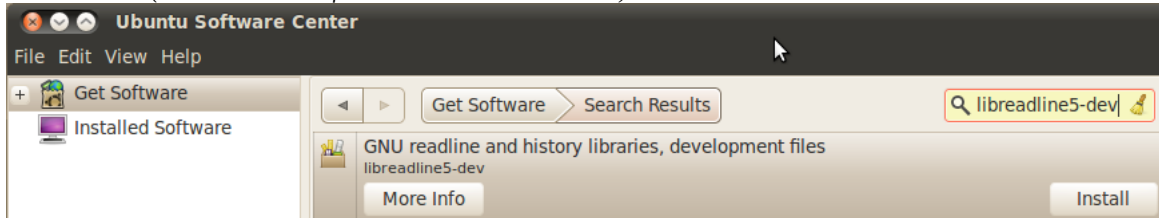
You can find more detailed information about setting up your own repository here [Working with CVS](#).

Configure source code

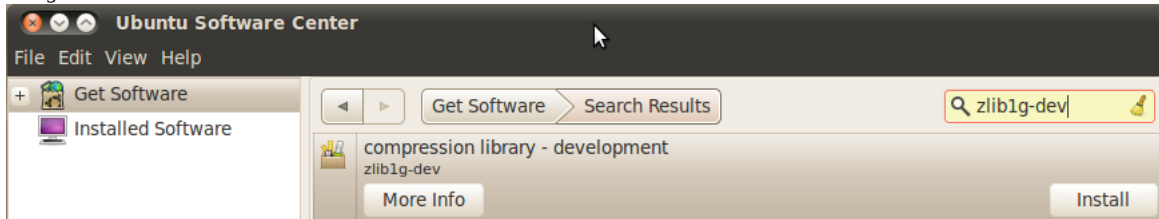
Install components with Software-Center

You need the following Software, and you can download them through Software Center (*Applications » Ubuntu Software Center*):

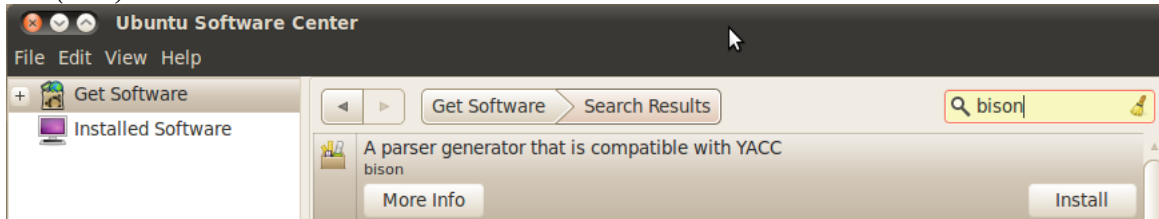
libreadline5-dev (For ubuntu 14.04 software version is libreadline6-dev)



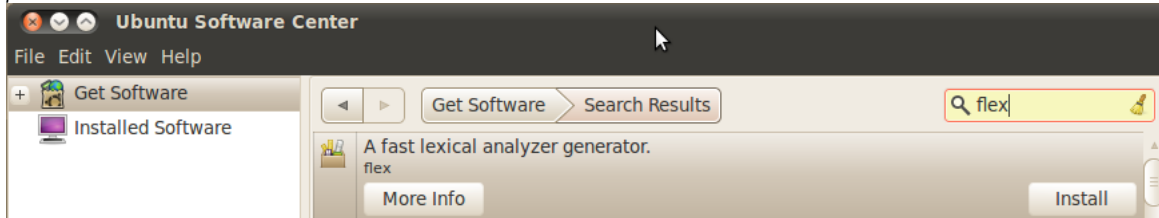
zlib1g-dev



bison (YACC)



flex



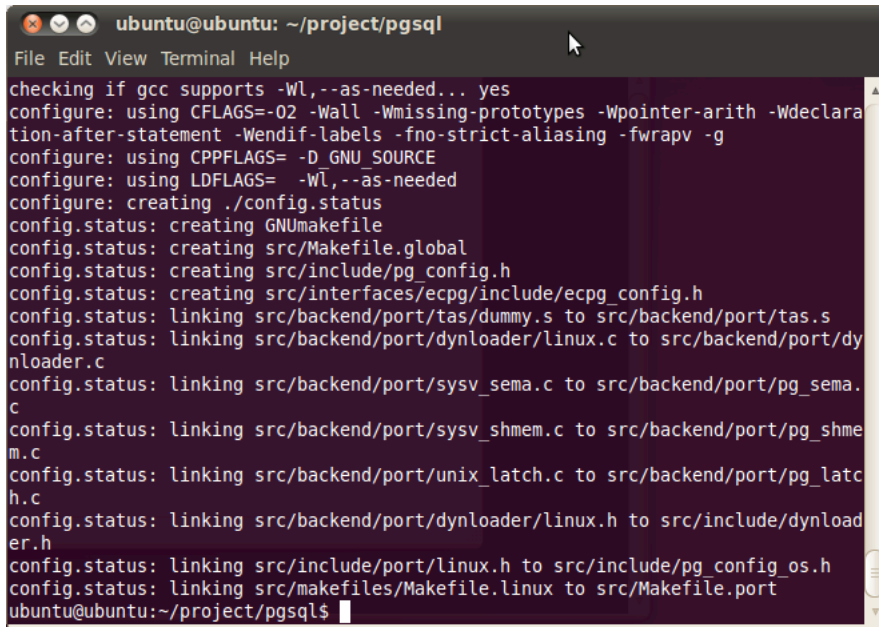
You simply have to type the names of the software in the search area, and install them. This is the list of the required Software:

1. *libreadline5-dev*, 2. *zlib1g-dev*, 3. *bison (YACC)*, 4. *flex*

Configure in the terminal

Switch back to the terminal and execute the following commands:

```
cd pgsq1
./configure --prefix=$HOME/project --enable-depend --enable-cassert --enable-debug
```



```

ubuntu@ubuntu: ~/project/pgsql
File Edit View Terminal Help
checking if gcc supports -Wl,-as-needed... yes
configure: using CFLAGS=-O2 -Wall -Wmissing-prototypes -Wpointer-arith -Wdeclaration-after-statement -Wendif-labels -fno-strict-aliasing -fwrapv -g
configure: using CPPFLAGS= -D GNU_SOURCE
configure: using LDFLAGS= -Wl,-as-needed
configure: creating ./config.status
config.status: creating GNUmakefile
config.status: creating src/Makefile.global
config.status: creating src/include/pg_config.h
config.status: creating src/interfaces/ecpg/include/ecpg_config.h
config.status: linking src/backend/port/tas/dummy.s to src/backend/port/tas.s
config.status: linking src/backend/port/dynloader/linux.c to src/backend/port/dynloader.c
config.status: linking src/backend/port/sysv_sema.c to src/backend/port/pg_sema.c
config.status: linking src/backend/port/sysv_shmem.c to src/backend/port/pg_shmem.c
config.status: linking src/backend/port/unix_latch.c to src/backend/port/pg_latch.c
config.status: linking src/backend/port/dynloader/linux.h to src/include/dynloader.h
config.status: linking src/include/port/linux.h to src/include/pg_config_os.h
config.status: linking src/makefiles/Makefile.linux to src/Makefile.port
ubuntu@ubuntu:~/project/pgsql$

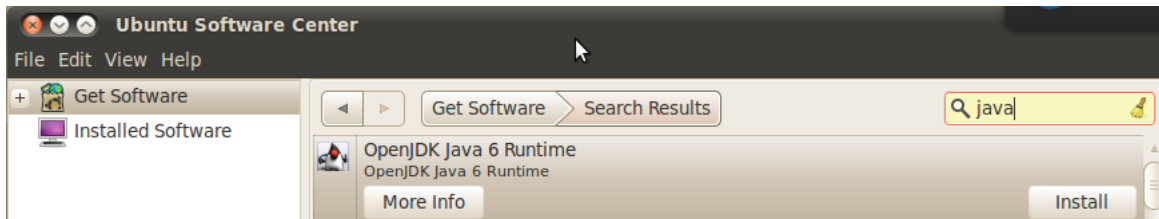
```

Download Eclipse

Install JDK for Eclipse

To see the PostgreSQL source code in an organized manner we will install Eclipse on the computer.
(Remark: You can use any other development tool also, but this manual explains the usage with Eclipse).

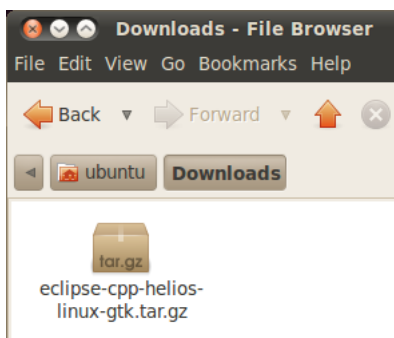
First of all you need the "OpenJDK Java 6 Runtime" for Eclipse, which you can also download in the Software-Center.
Start Software Center Center (*Applications » Ubuntu Software Center*) or switch to the opened window and search for java. Choose "OpenJDK Java 6 Runtime" and click **Install**.



Download Eclipse

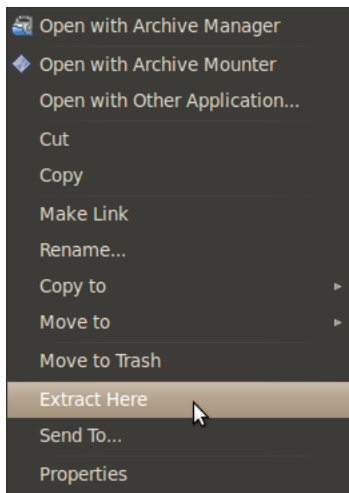
You can download Eclipse from the following link: <http://www.eclipse.org/downloads/packages/eclipse-ide-cc-developers/heliosr>

It's important that you choose a **Helios Version**, which supports C/C++. Open the Folder in your file browser, where the tar file is. In this example: *ubuntu » Downloads*.

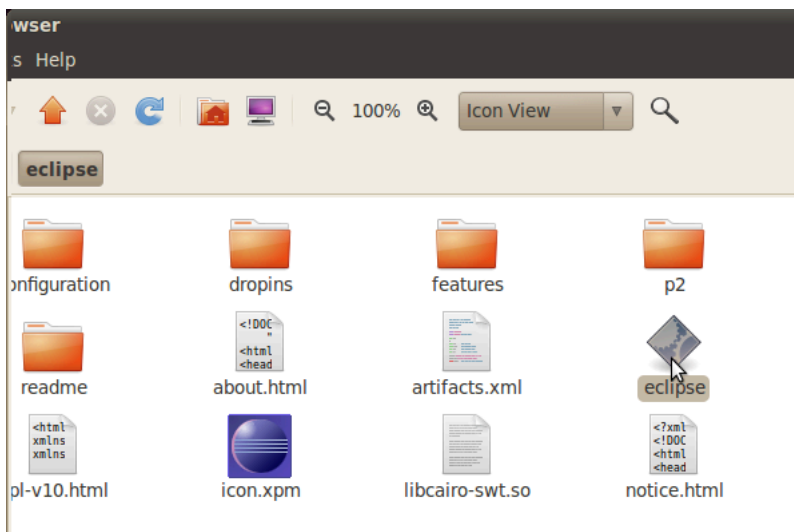


Extract and start

Right click on the Package and choose "Extract Here" from the menu.



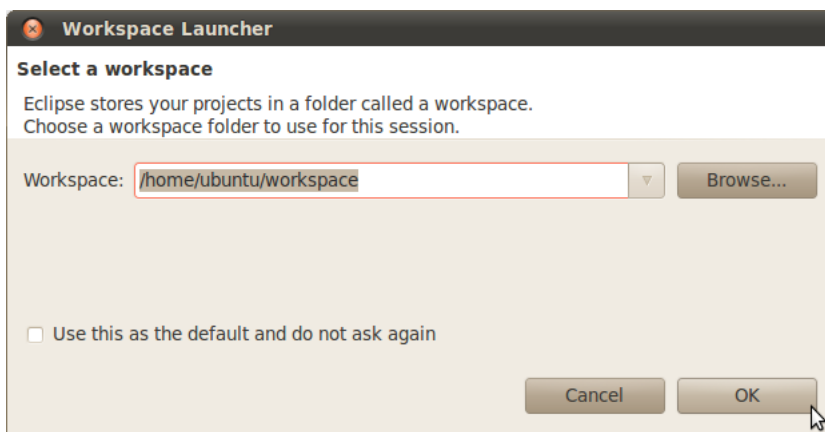
Now enter the extracted Folder and double-click the file "eclipse".



Set up Workspace

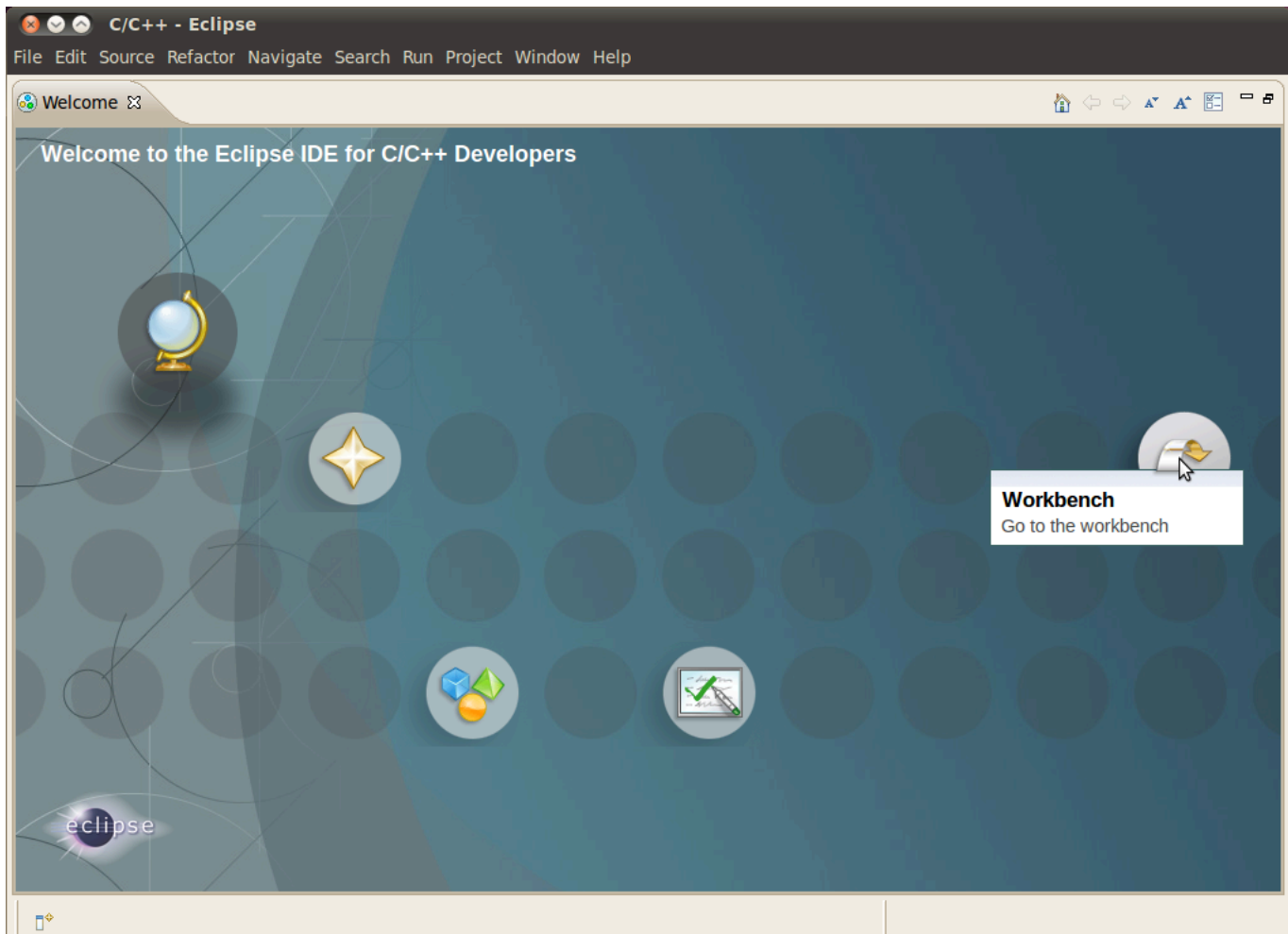
You should set up a workspace, where the software stores your projects:

After starting the program you will be prompted to select a folder for your workspace. We will choose the recommended path, but you can change it if you browse for another folder. If you have chosen your Folder click "OK".

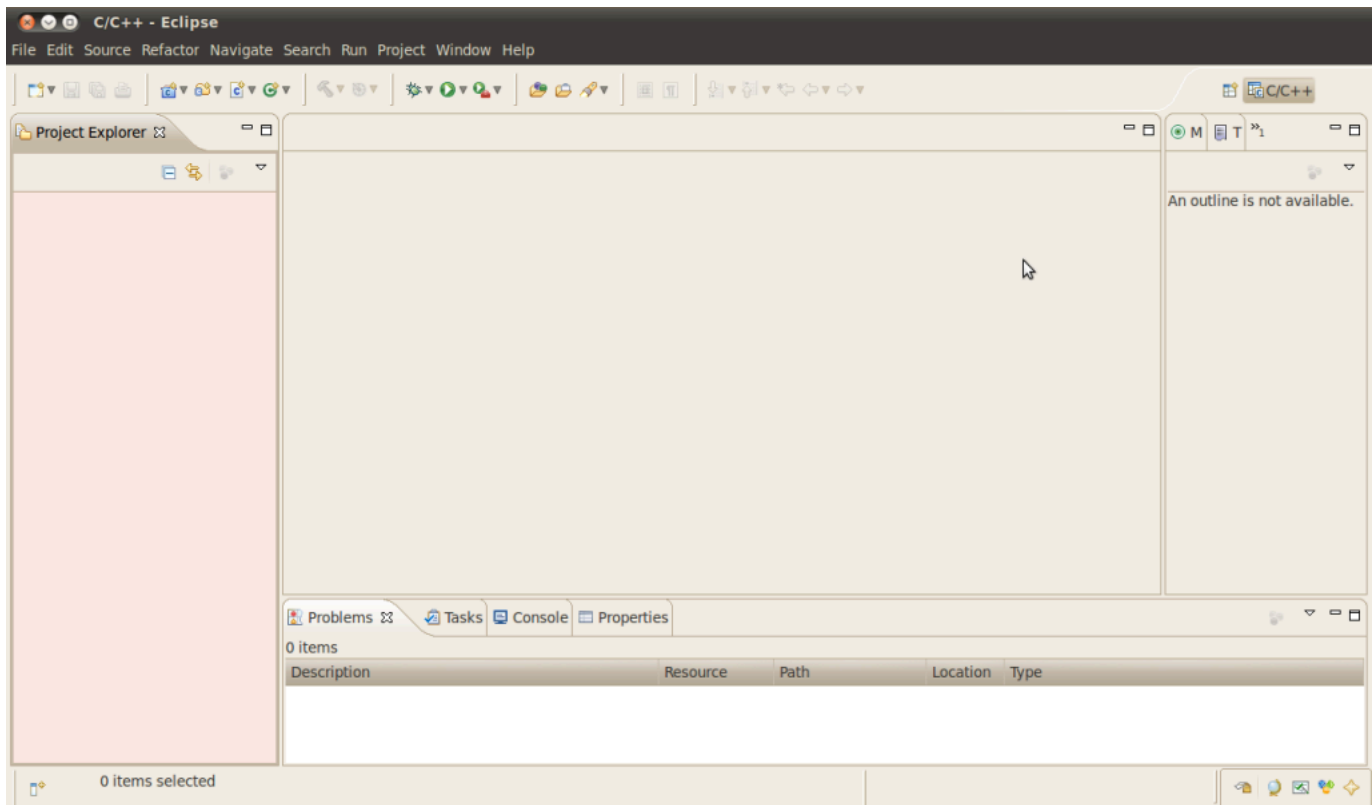


The program

Now you see the welcome-page of Eclipse, where you can choose tutorials, if you are not familiar with Eclipse. To continue the manual choose the arrow on the right to open your Workbench (the defaults layout of the program).

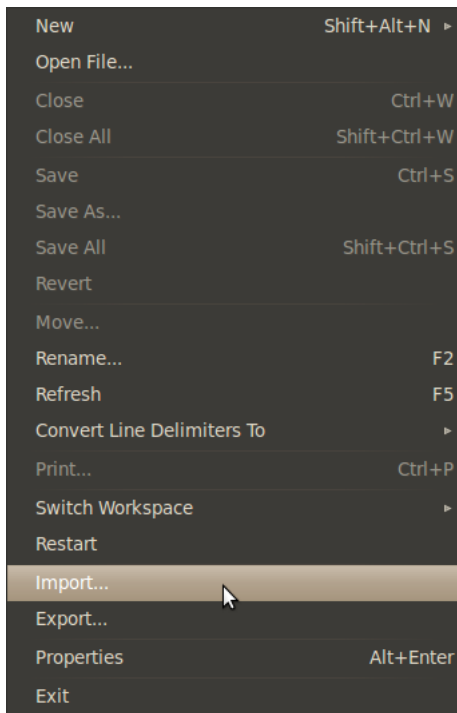


We will import the PostgreSQL code in this workbench.

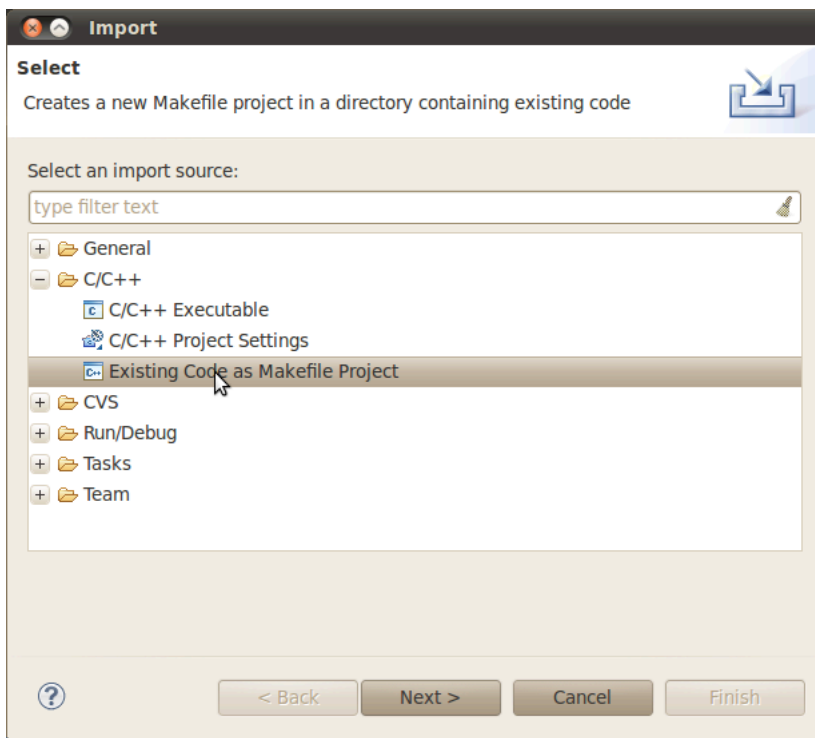


Import

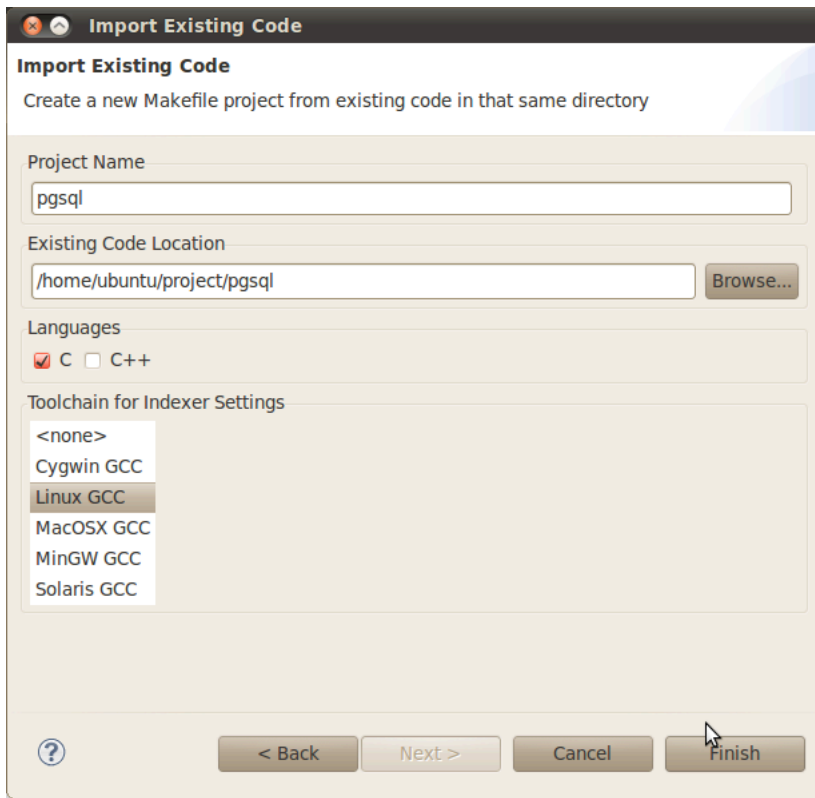
In this step we will import the project. Select in the left top of the program "File » Import "



In the new window select "C/C++ » Existing Code as Makefile Project" and click the "**next**" button:

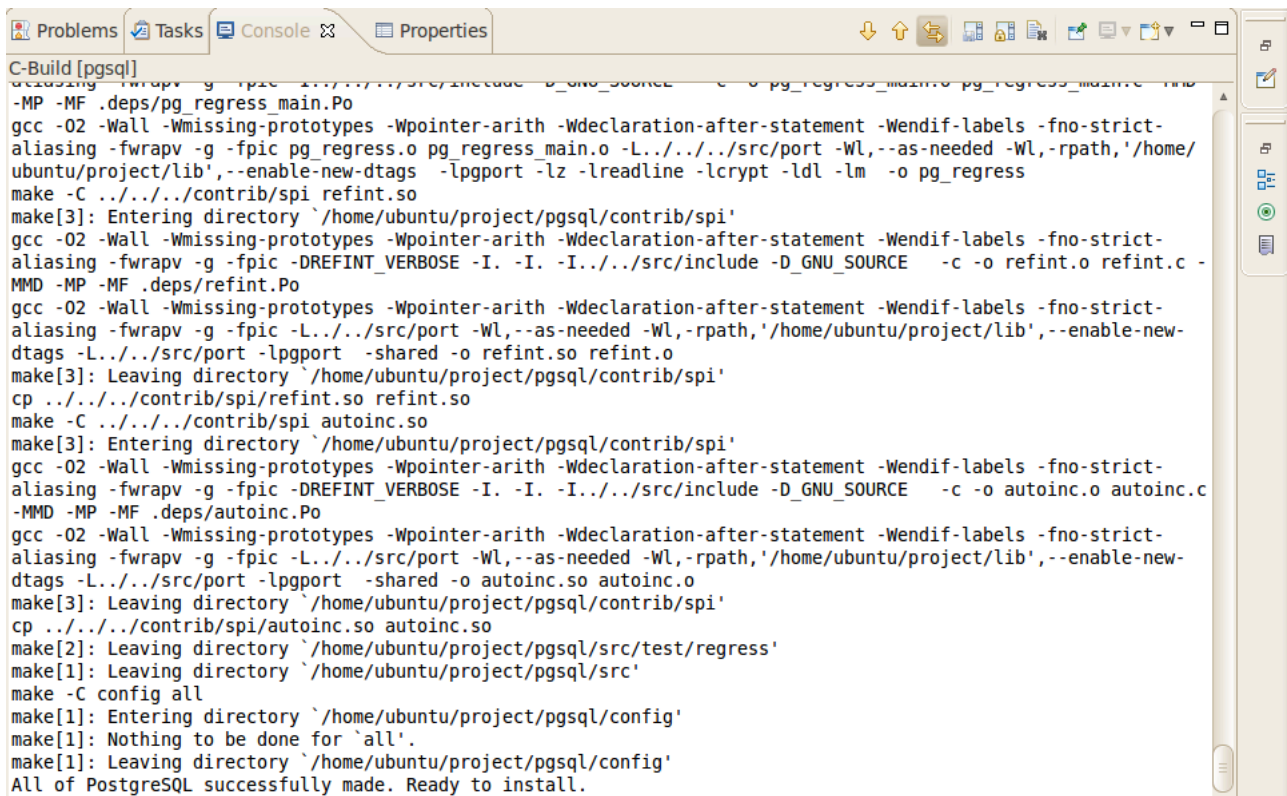


In the next window click browse, and choose the **pgsql** folder. "Language" is "C" and "Toolchain for Indexer Settings" select "**Linux GCC**". At the end click **finish**.



"Make" will start automatically (this can take several minutes). If every components work well you should have the following message in you console:
"All of PostgreSQL successfully made, Ready to install"

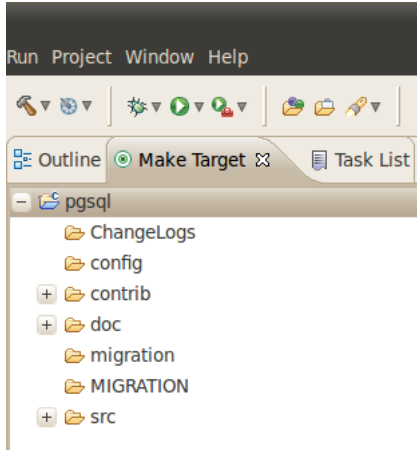
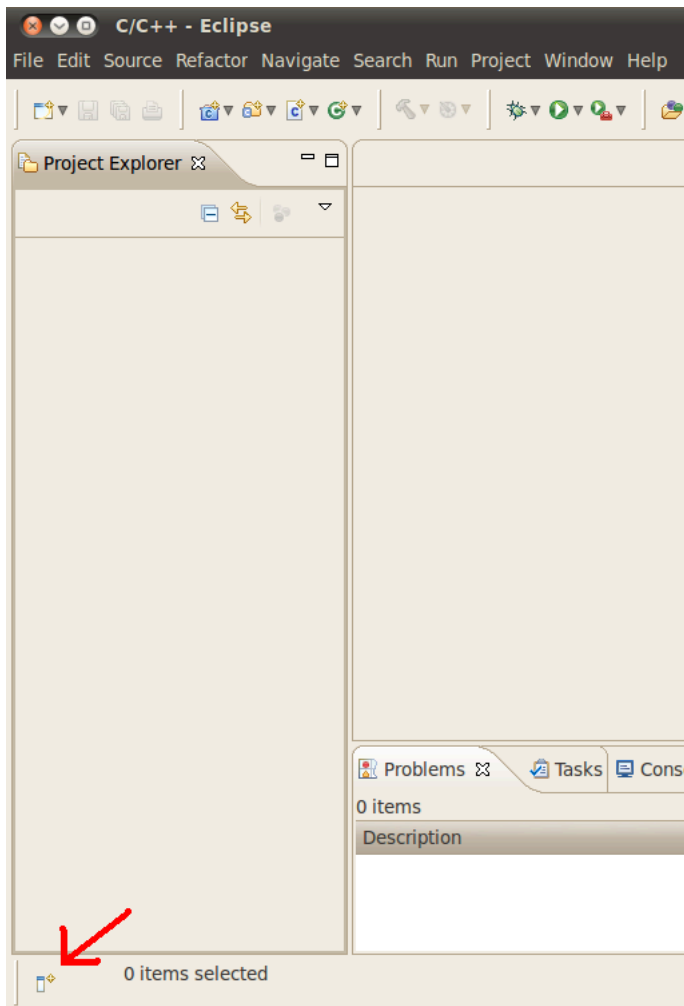
Furthermore you can see the Project "pgsql" on the left side of your workbench in the project explorer view.



Adding make targets

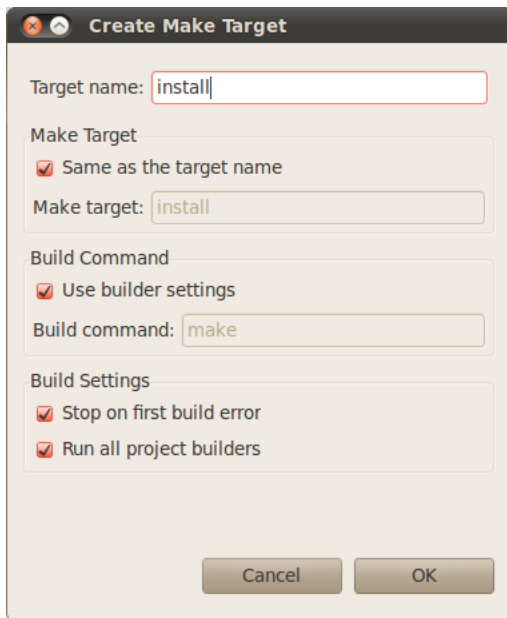
Open the Make Target view

If this you can't find this view, simply click on the Button in the left bottom to add the.

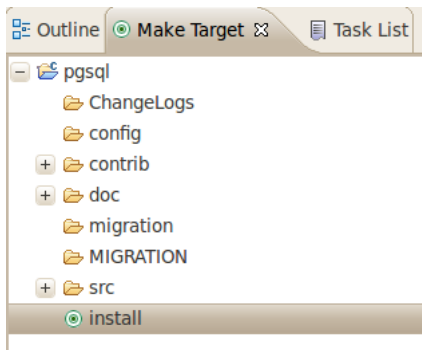


Add Target

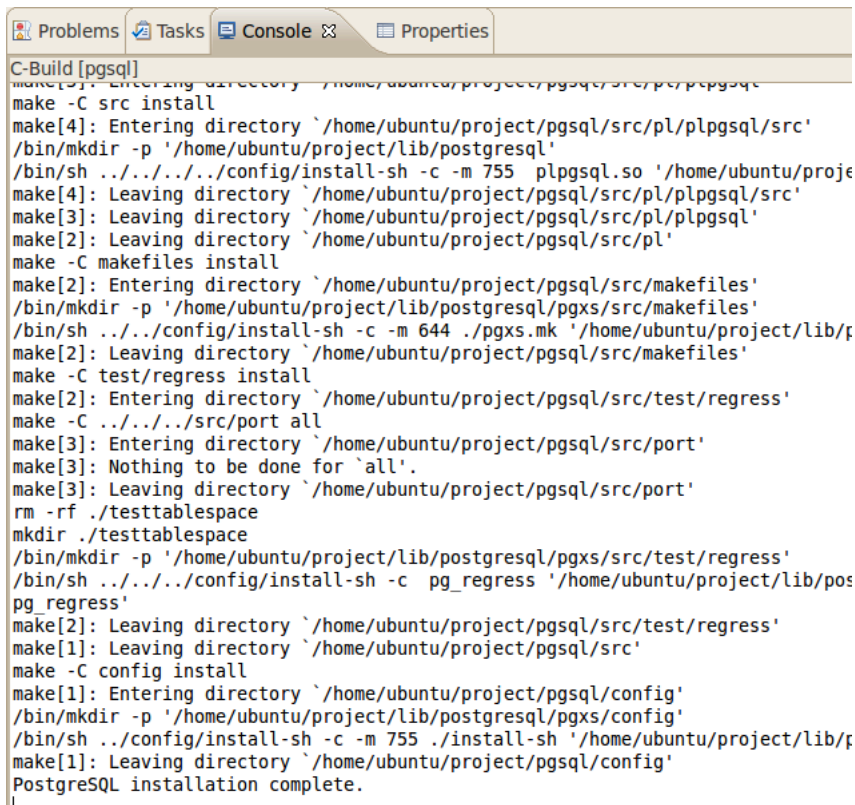
In the *Make-Target-view* right click on the project name and select "New". In the new window "*Create Make Target*" type at "*Target name:*" "**install**" and click "**OK**".



If this was successful in the *Make-Target-view* at the end of the list a green dot appears with the label `install`. Double-click this, so you start the installation of PostgreSQL.



If the installation was successful in the console you can read:
“PostgreSQL installation complete”.



```

C-Build [pgsql]
make[3]: Entering directory `/home/ubuntu/project/pgsql/src/plpgsql'
make -C src install
make[4]: Entering directory `/home/ubuntu/project/pgsql/src/plpgsql/src'
/bin/mkdir -p '/home/ubuntu/project/lib/postgresql'
/bin/sh ../../../../config/install-sh -c -m 755 plpgsql.so '/home/ubuntu/proje
make[4]: Leaving directory `/home/ubuntu/project/pgsql/src/plpgsql/src'
make[3]: Leaving directory `/home/ubuntu/project/pgsql/src/plpgsql'
make[2]: Leaving directory `/home/ubuntu/project/pgsql/src/pl'
make -C makefiles install
make[2]: Entering directory `/home/ubuntu/project/pgsql/src/makefiles'
/bin/mkdir -p '/home/ubuntu/project/lib/postgresql/pgxs/src/makefiles'
/bin/sh ../../../../config/install-sh -c -m 644 ./pgxs.mk '/home/ubuntu/project/lib/p
make[2]: Leaving directory `/home/ubuntu/project/pgsql/src/makefiles'
make -C test/regress install
make[2]: Entering directory `/home/ubuntu/project/pgsql/src/test/regress'
make -C ../../../../src/port all
make[3]: Entering directory `/home/ubuntu/project/pgsql/src/port'
make[3]: Nothing to be done for 'all'.
make[3]: Leaving directory `/home/ubuntu/project/pgsql/src/port'
rm -rf ./testtablespace
mkdir ./testtablespace
/bin/mkdir -p '/home/ubuntu/project/lib/postgresql/pgxs/src/test/regress'
/bin/sh ../../../../config/install-sh -c pg_regress '/home/ubuntu/project/lib/pos
pg_regress'
make[2]: Leaving directory `/home/ubuntu/project/pgsql/src/test/regress'
make[1]: Leaving directory `/home/ubuntu/project/pgsql/src'
make -C config install
make[1]: Entering directory `/home/ubuntu/project/pgsql/config'
/bin/mkdir -p '/home/ubuntu/project/lib/postgresql/pgxs/config'
/bin/sh ../config/install-sh -c -m 755 ./install-sh '/home/ubuntu/project/lib/p
make[1]: Leaving directory `/home/ubuntu/project/pgsql/config'
PostgreSQL installation complete.

```

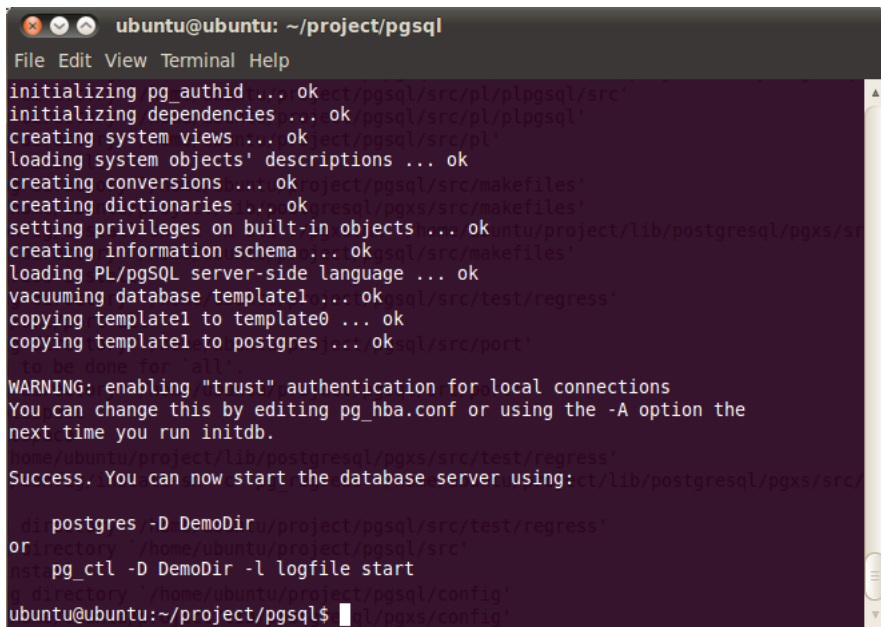
Launch initdb in the terminal

Switch back to the terminal and change to the path `"ubuntu/project/pgsql"`. Before starting initdb you should set some environment variables.

```

export PATH=$HOME/project/bin:$PATH
export PGDATA=DemoDir
initdb

```



```

ubuntu@ubuntu: ~/project/pgsql
File Edit View Terminal Help
initializing pg_authid ... ok
initializing dependencies ... ok
creating system views ... ok
loading system objects' descriptions ... ok
creating conversions ... ok
creating dictionaries ... ok
setting privileges on built-in objects ... ok
creating information schema ... ok
loading PL/pgSQL server-side language ... ok
vacuuming database template1 ... ok
copying template1 to template0 ... ok
copying template1 to postgres ... ok
Nothing to be done for 'all'.
WARNING: enabling "trust" authentication for local connections
You can change this by editing pg_hba.conf or using the -A option the
next time you run initdb.
/home/ubuntu/project/lib/postgresql/pgxs/src/test/regress'
Success. You can now start the database server using:
psql postgres -D DemoDir
or
pg_ctl -D DemoDir -l logfile start
ubuntu@ubuntu:~/project/pgsql$

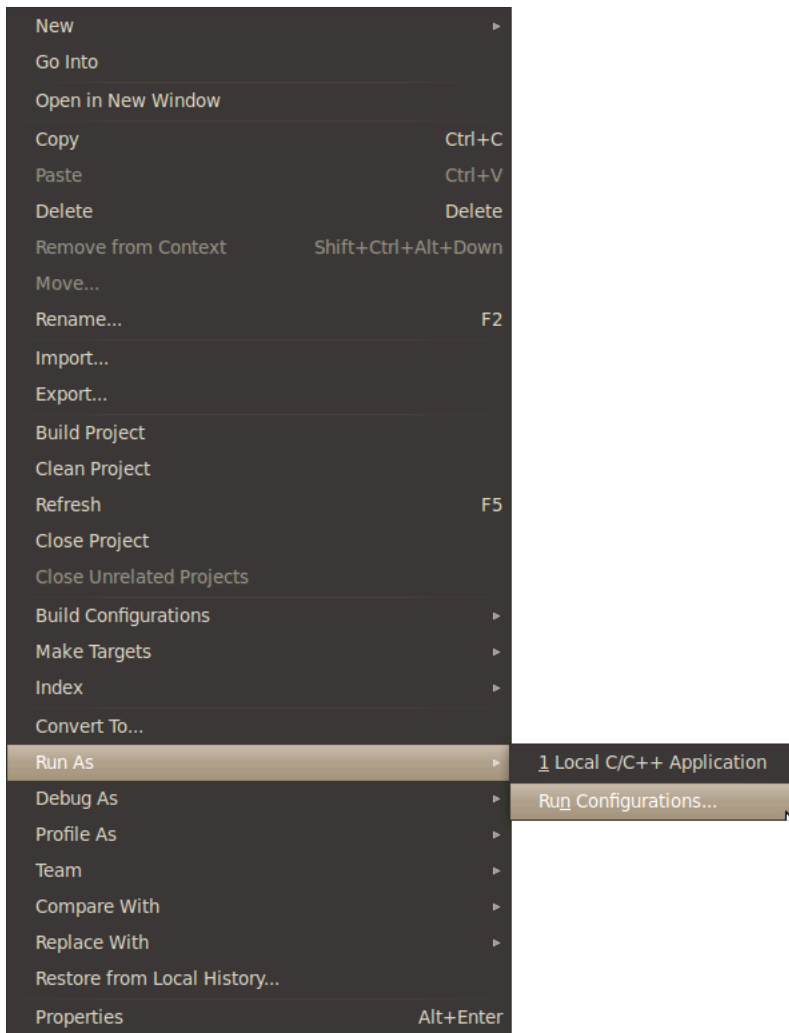
```

Run Configurations in Eclipse

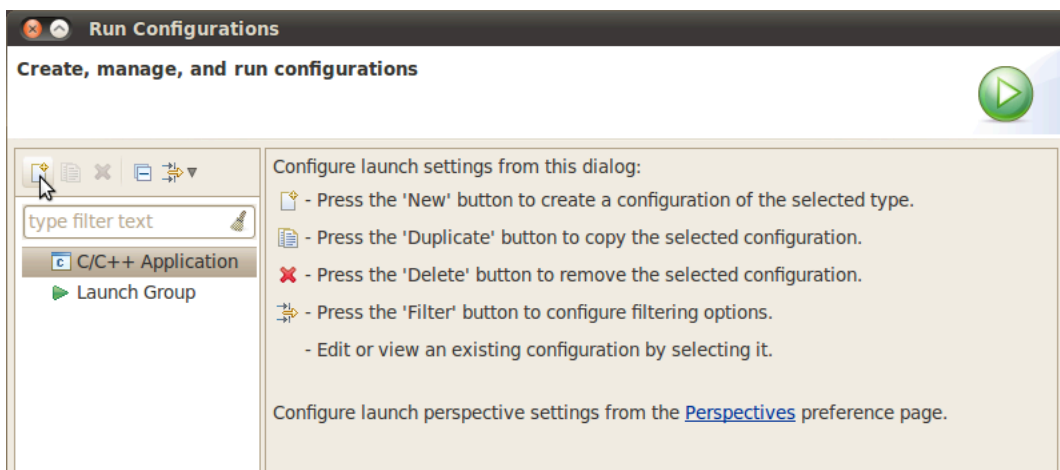
Switch back to Eclipse.

Select the Project Explorer view. If you can't see it, add it with the button in the left bottom.

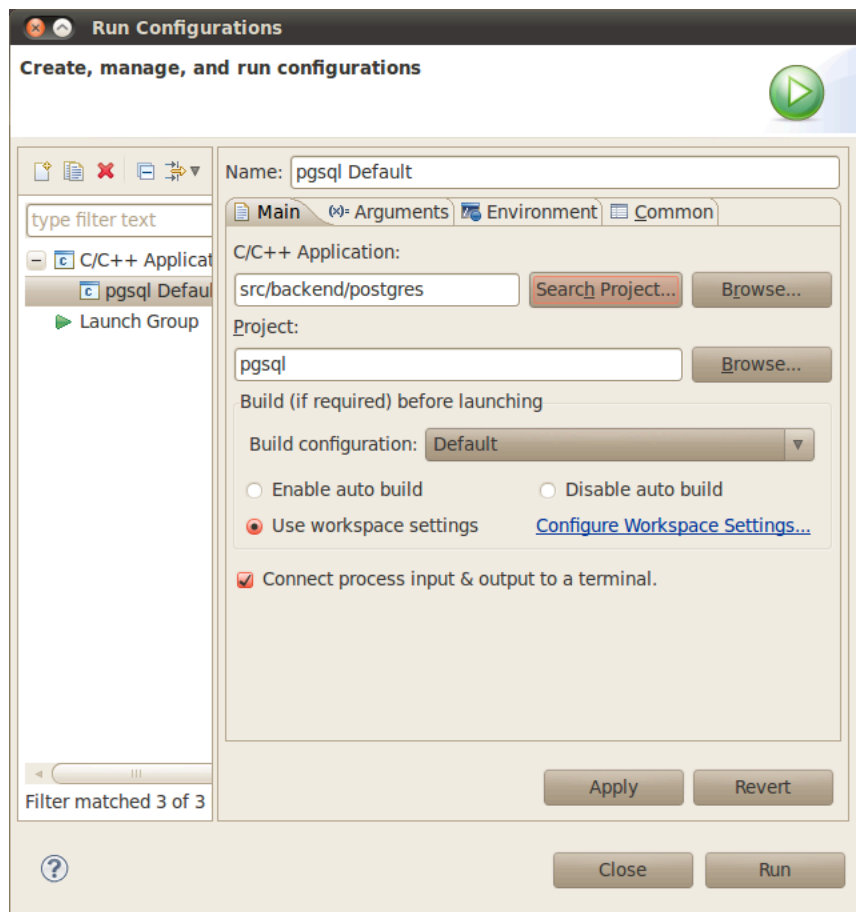
In the Project Explorer view right click on the Projektname (pgsql), and select "Run As » Run Configurations".



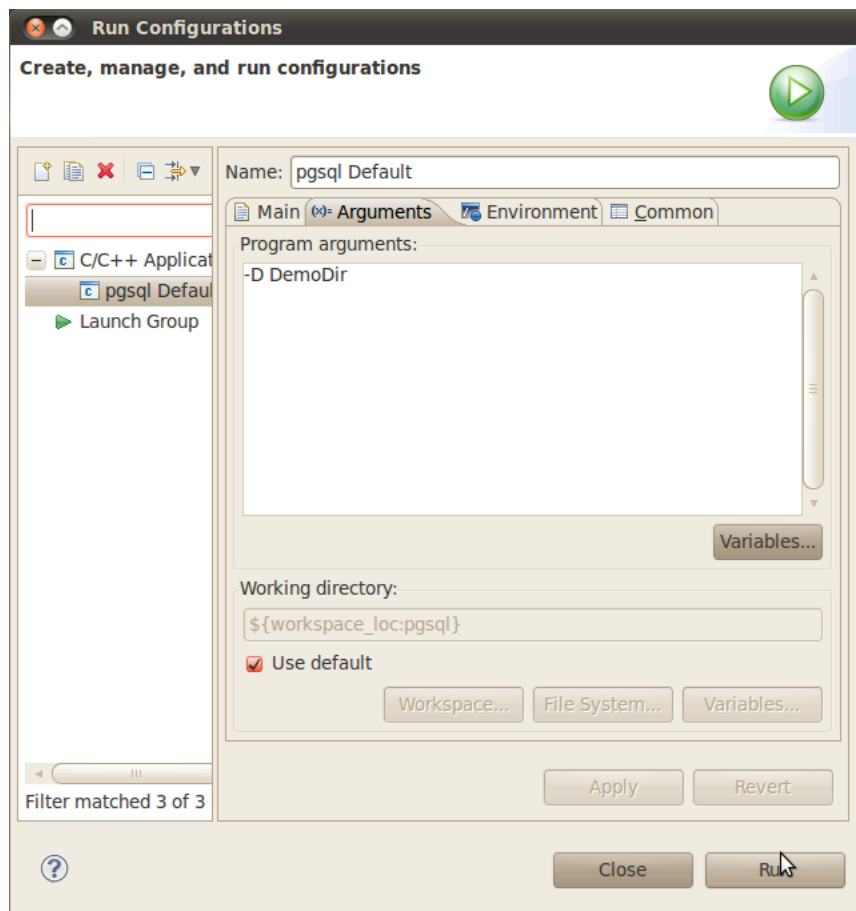
In this Window you can add a configuration:



Double-click "*C/C++ Application*" so a subfolder is created with the name "*pgsql-Default*". Now fill in the form on the right side. At "*C/C++ Application*:" the path: "*src/backend/postgres*" and project should be: "**pgsql**".



Switch to the tab "Arguments". At "Program arguments" type the arguments "-D DemoDir" and click the "Apply" button. Click "Run" to start the program.



You should see following logs in your console:

LOG: database system was shut down at (current date and time) UTC

LOG: database system is ready to accept connections

LOG: autovacuum launcher started

```

pgsql Default [C/C++ Application] /home/ubuntu/project/pgsql/src/backend/postgres
LOG: database system was shut down at 2010-10-06 19:35:34 UTC
LOG: database system is ready to accept connections
LOG: autovacuum launcher started

```

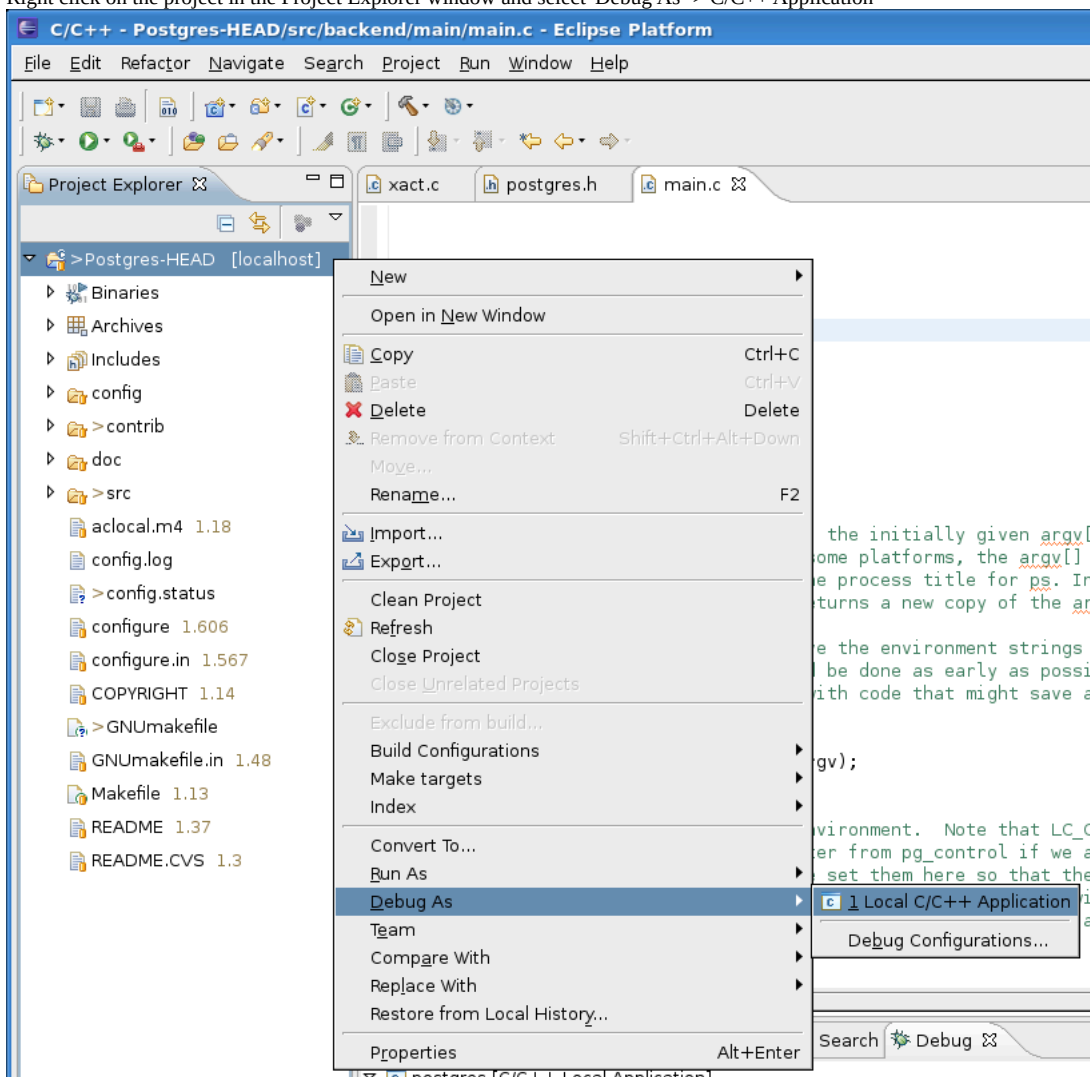
That means, the installation was successful and the server is running!

Add include files

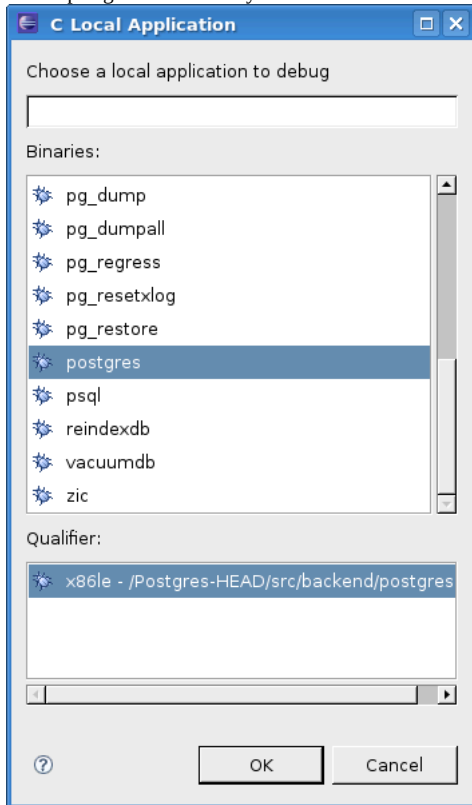
When looking at source code files, you will see compiler errors saying that include files were not found. To fix this, you need to add and include directory to properties. From the menus, choose Project/Properties/ C/C++ General / Paths and Symbols/Includes. Add an Include directory. Click on Workspace. Choose <project>/src/include and hit enter. Allow an index rebuild.

Debugging PostgreSQL from Eclipse

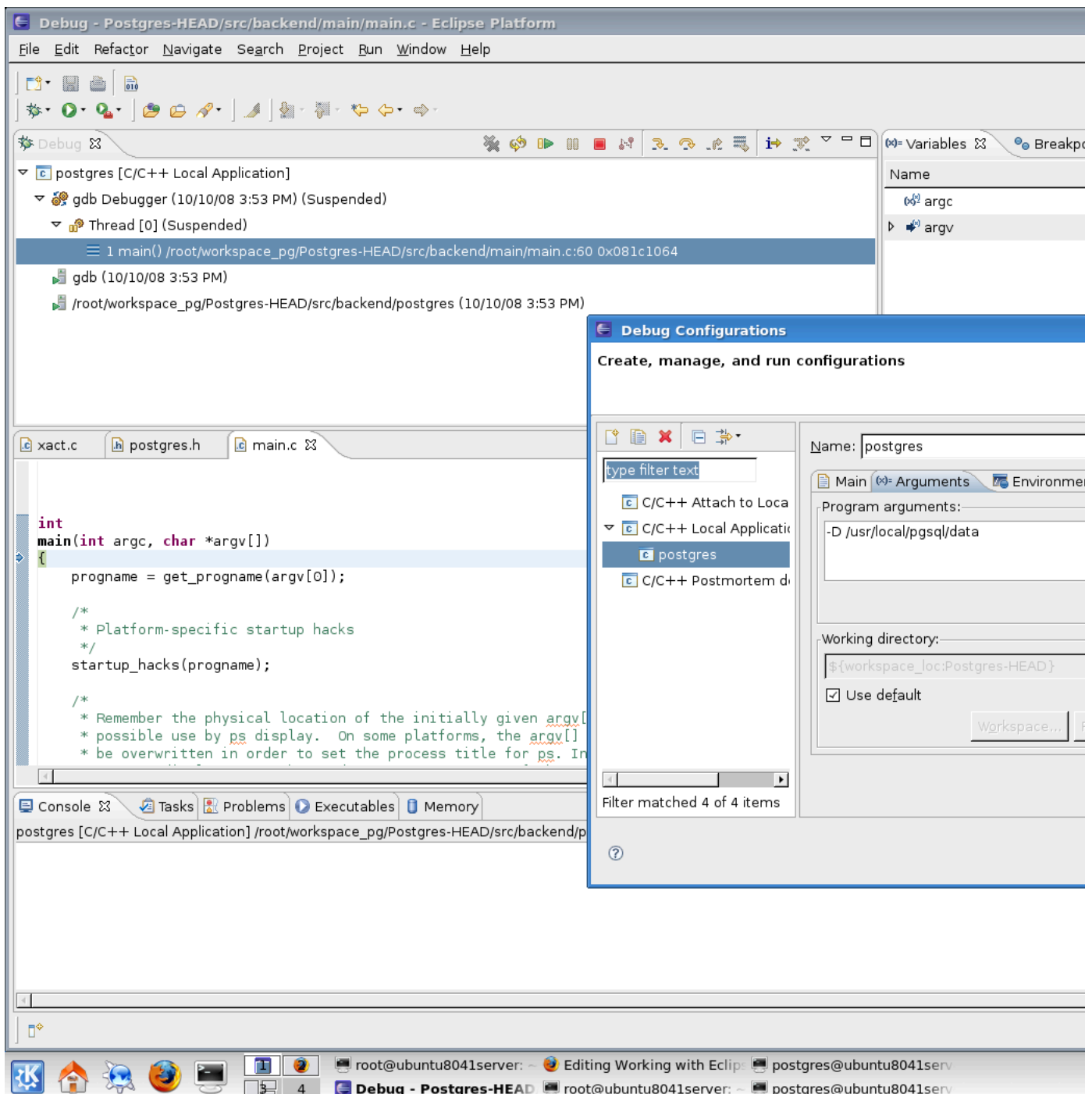
1. Make sure gdb is installed on your machine
2. Right click on the project in the Project Explorer window and select 'Debug As -> C/C++ Application'



3. Select 'postgres' as the binary to execute



4. If you don't have PGDATA set in the environment Eclipse is running in, you can tune the command line arguments in Run/Debug Configurations so that Postgres can start properly



Tip: You can do a 'make install' and make your arguments points to that installed directory.

Debugging with child processes

When you start debugging process by using the project default, it can only debug the postmaster process. When clients connect to the database, the postmaster process forks child processes, each of which is actually postgres backend process responsible for the response to the client. You might want gdb to be able to access the forked processes too. Here's how to set that up:

1. Debug Perspective => Run => Debug Configuration
2. On left side of menu you can see "C/C++ Attach to Application"
3. Right click on "C/C++ Attach to Application" and create new debug configuration,
4. Set Project, Build Configuration as "Use Active" , C/C++ Application as "postgres" executable path,
5. Start postmaster & one instance of postgresql client in the terminal(for creating one new postgres),
6. Click on Debug which will show current process list,
7. Select forked "postgres" process you want to debug
8. Put breakpoint in your function,

Note: if attaching to a process fails while logged on as an ordinary user, it may be due to Maverick Meerkat (10.10) Ubuntu introducing a patch to disallow ptracing of non-child processes by non-root users - ie. only a process which is a parent of another process can ptrace it for normal users - whilst root can still ptrace every process. Hence why you can use gdb to attach via sudo still.

You can temporarily disable this restriction (and revert to the old behaviour allowing your user to ptrace (gdb) any of their other processes) by executing:

```
echo 0 | sudo tee /proc/sys/kernel/yama/ptrace_scope
```

To permanently allow it edit /etc/sysctl.d/10-pttrace.conf and change the line:

```
kernel.yama.pttrace_scope = 1
```

to

```
kernel.yama.pttrace_scope = 0
```

Create and open Database

To see, that the installation is completely working you can do the followings to test the database.

Switch to the terminal, and change to the path "ubuntu/project/pgsql". Execute the command:

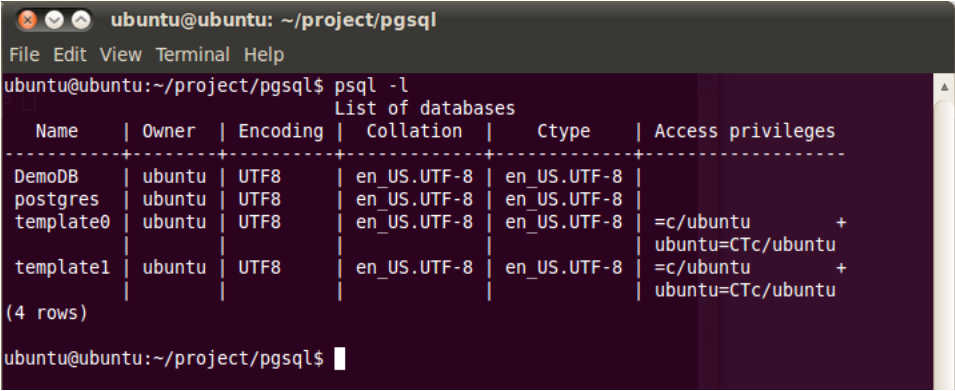
```
psql -l
```

This will list you all available database.

To create your own database simply type the following:

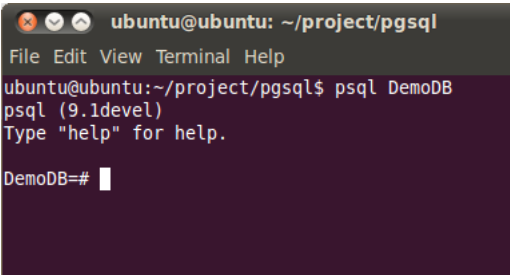
```
createdb DemoDB
```

If you execute the list-database again (psql -l) you can see your database, "DemoDB":



You can login to your database with:

```
psql DemoDB
```



Now you can create tables, relationships and much more. You can fully access your database. To exit your database type:

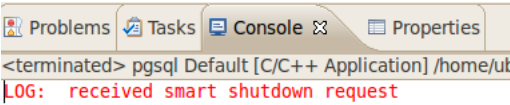
```
/q
```

Terminate the program

The server is available until you click on the red rectangle in the console of Eclipse.

If the stop was successful you will see the following log in the console:

LOG: received smart shutdown request



Retrieved from "https://wiki.postgresql.org/index.php?title=Working_with_Eclipse&oldid=32917"

■ This page was last edited on 28 December 2018, at 22:32.